

Universidad del Aconcagua

Facultad de Ciencias Sociales y Administrativas

Sistema de Gestión Universitaria

Programación Avanzada — Proyecto Final

Integrantes:

Axel Sarmiento

Julián Molina

Andrés Riveros

17 de junio de 2026

Repositorio: https://github.com/AxelMrak/proyecto_final_prog_avanzada

Índice

1. Resumen	3
2. Introducción	3
2.1. Contexto	3
2.2. Objetivos	3
2.3. Alcance	4
3. Descripción del Proyecto	4
3.1. Concepto	4
3.2. Funcionalidades	4
3.3. Capturas del sistema	5
4. Tecnologías Utilizadas	6
5. Arquitectura del Software	7
5.1. Arquitectura limpia en cuatro capas	7
5.2. Diagrama de capas	7
5.3. Diagrama UML de clases	7
6. Estructura del Proyecto	8
7. Detalle de Implementación	10
7.1. Capa de dominio: Value Object Nota	10
7.2. Validación por campo con <code>InputReader</code>	10
7.3. Persistencia con repositorios JDBC	11
7.4. Migrador de base de datos	12
7.5. Caso de uso: carga de nota	12
8. Patrones de Diseño Aplicados	13
9. Instrucciones de Ejecución	14
9.1. Requisitos	14
9.2. Compilación	14
9.3. Ejecución	15
9.4. Ejecución de tests de integración	15
10. Evidencias del Proyecto	15
10.1. Video de demostración de la aplicación	15
10.2. Video de recorrido por el código y estructura	15
10.3. Diagrama de arquitectura	16

10.4. Contenido de la carpeta Recursos/	17
11.Pruebas y Verificación	17
11.1. Compilación	17
11.2. Test de integración	17
11.3. Smoke test manual	18
12.Conclusiones	18
13.Trabajo Futuro	18

1. Resumen

El presente informe documenta el desarrollo del **Sistema de Gestión Universitaria**, una aplicación de consola en Java 17 que administra alumnos, docentes, administrativos, materias e inscripciones de una universidad. La aplicación permite dar de alta usuarios, inscribir alumnos en materias, y que los docentes carguen notas finales, todo respaldado por una base de datos SQLite con esquema normalizado.

El proyecto se destaca por haber sido diseñado desde el inicio con una *arquitectura limpia* en cuatro capas — dominio, aplicación, infraestructura y presentación — y por aplicar los principios **SOLID** de forma deliberada. La capa de dominio está completamente aislada de detalles de infraestructura (JDBC, SQLite), las interfaces de repositorio respetan el *Interface Segregation Principle*, y la inyección de dependencias se realiza manualmente en el punto de entrada.

El sistema se ejecuta desde terminal, con un menú interactivo que valida cada campo en el momento del ingreso y ofrece listas numeradas para seleccionar alumnos y materias sin necesidad de memorizar legajos o códigos.

Palabras clave: Java 17, SQLite, JDBC, arquitectura limpia, SOLID, Maven, aplicación de consola.

2. Introducción

2.1. Contexto

La cátedra de **Programación Avanzada** propone como trabajo final el desarrollo de un sistema de información en Java que aplique los conceptos de la materia: diseño orientado a objetos, persistencia con base de datos, separación de responsabilidades, principios SOLID y documentación profesional. El equipo seleccionó un sistema de gestión universitaria por su riqueza de entidades (usuarios con distintos roles, materias, inscripciones, notas) y porque permite demostrar la aplicación de patrones de diseño en un dominio realista.

2.2. Objetivos

- Implementar un sistema de consola funcional para la gestión de alumnos, docentes, administrativos, materias e inscripciones.
- Diseñar una **arquitectura en capas** que separe el dominio de la infraestructura y de la presentación.
- Normalizar el esquema de base de datos a tercera forma normal (3NF) con restricciones CHECK e índices sobre claves foráneas.

- Aplicar los principios **SOLID**, en particular SRP, ISP y DIP.
- Validar cada entrada del usuario en el momento del ingreso, sin postergar errores al final del formulario.
- Entregar un informe profesional y un repositorio público con control de versiones.

2.3. Alcance

El proyecto comprende:

- El código fuente completo del sistema (33 archivos Java).
- Scripts SQL de migración (DDL + datos semilla).
- Un test de integración que verifica 25 casos (validadores, constructores, carga de nota, persistencia).
- El presente informe en $\text{\LaTeX}$ (compilable a PDF).
- Repositorio remoto: https://github.com/AxelMrak/proyecto_final_prog_avanzada.
- Videos demostrativos y capturas de pantalla en la carpeta **Recursos/**.

3. Descripción del Proyecto

3.1. Concepto

El **Sistema de Gestión Universitaria** es una aplicación de línea de comandos que modela las operaciones habituales de una facultad. Un menú principal da acceso a tres submódulos: Alumnos, Docentes y Administrativos.

3.2. Funcionalidades

- **Alta de usuarios:** registro de alumnos (con carrera y año de ingreso), docentes (con departamento y categoría) y administrativos (con sector).
- **Listado:** consulta tabulada de cada tipo de usuario, con sus datos completos.
- **Inscripción a materias:** desde el menú de Alumnos, se selecciona un alumno (por legajo o eligiendo de una lista numerada) y una materia (por código o lista), y se registra la inscripción.

- **Carga de notas:** desde el menú de Docentes, se selecciona un alumno y una materia de listas desplegables, se ingresa una nota (0–10) y se persiste en la base de datos. El sistema valida que el alumno exista, que la materia exista y que el alumno esté inscripto en ella.
- **Validación por campo:** cada campo (nombre, legajo, email, año, nota) se valida en el momento de ingreso; si falla, se re-pregunta el mismo campo sin perder los anteriores.

3.3. Capturas del sistema

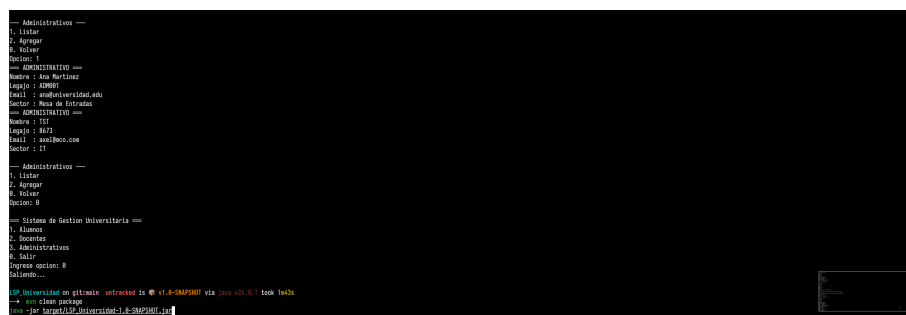


Figura 1: Menú principal y listado de alumnos.

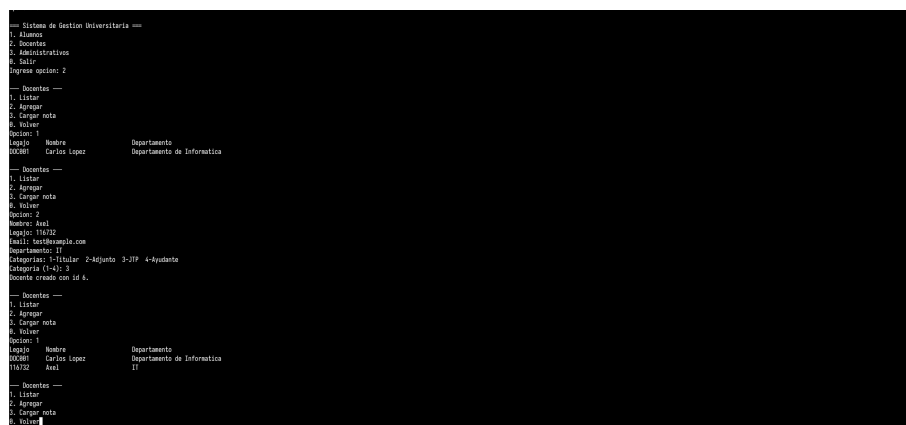


Figura 2: Inscripción de un alumno a una materia: selección por lista numerada.

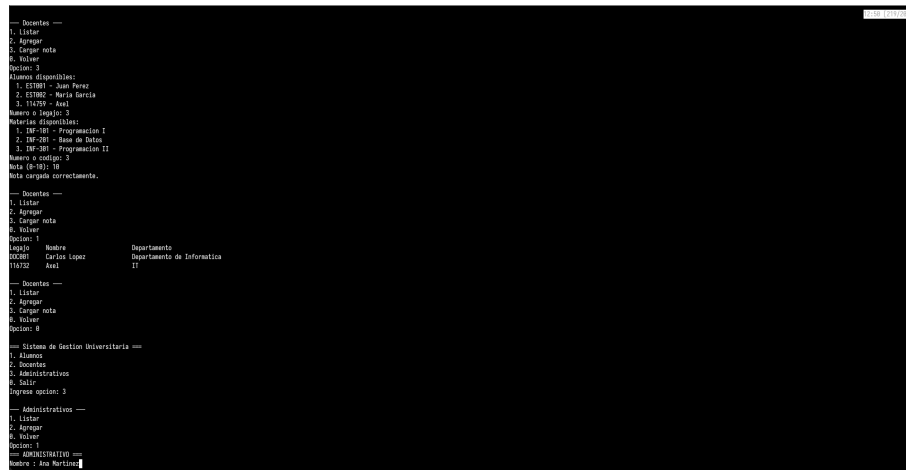


Figura 3: Carga de nota: el docente selecciona alumno y materia de listas desplegables e ingresa la nota.

4. Tecnologías Utilizadas

- **Java 17** — lenguaje de programación principal. Se usa `var` para inferencia de tipos locales, `switch expressions`, `record` y `streams`.
- **Maven 3** — gestor de construcción y dependencias. El plugin *maven-shade-plugin* empaqueta un JAR auto-contenido (fat JAR) con todas las dependencias.
- **SQLite 3 (JDBC)** — motor de base de datos embebido. Cero configuración: la base se crea automáticamente al iniciar la aplicación.
- **Git** — control de versiones. Repositorio público en GitHub.
- **L^AT_EX** — sistema de composición tipográfica para este informe.

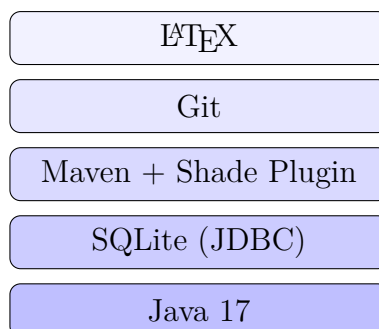


Figura 4: Stack tecnológico. Cada capa se apoya sobre la de abajo: la aplicación corre sobre Java, que accede a SQLite vía JDBC; Maven construye y empaqueta; Git versiona; L^AT_EX produce el informe.

5. Arquitectura del Software

5.1. Arquitectura limpia en cuatro capas

El sistema se organiza en cuatro capas con dependencias unidireccionales: la capa de dominio no conoce a ninguna otra; la de aplicación conoce al dominio; la de infraestructura implementa las interfaces del dominio; y la de presentación consume los servicios de aplicación.

- **domain/** — Entidades de negocio (`Estudiante`, `Docente`, `Administrativo`, `Materia`, `Inscripcion`), *value objects* (`Nota`), enumeraciones con id de base de datos (`Rol`, `CategoriaDocente`, `EstadoInscripcion`) y las interfaces de repositorio que definen *qué* operaciones de persistencia necesita el dominio, sin decir *cómo*.
- **application/** — Servicios de caso de uso: `EstudianteService`, `DocenteService`, `AdministrativoService`, `InscripcionService`. Orquestan las reglas de negocio (validar existencia, verificar inscripción previa, cargar nota) y delegan la persistencia a las interfaces del dominio.
- **infrastructure/** — Implementaciones concretas: `ConnectionFactory` (abre conexiones JDBC a SQLite), cinco repositorios JDBC que implementan las interfaces del dominio, y `Migrator` (ejecuta los scripts `.sql` en orden sobre el arranque).
- **ui/menu/** — Menús de consola (`MenuAlumnos`, `MenuDocentes`, `MenuAdministrativos`). Cada menú recibe sus dependencias por constructor y coordina la interacción con el usuario vía `InputReader`.

5.2. Diagrama de capas

5.3. Diagrama UML de clases

El siguiente diagrama muestra las entidades principales y sus relaciones.

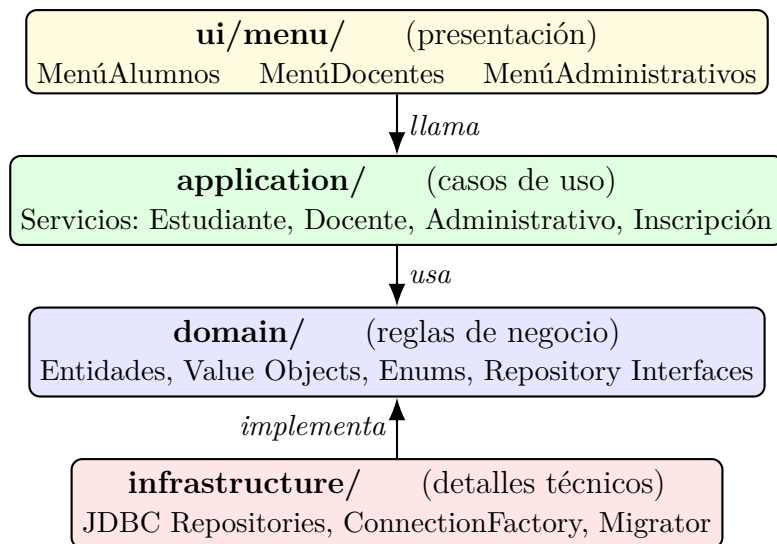


Figura 5: Las cuatro capas del sistema. La UI llama a los servicios de aplicación; éstos usan el dominio (entidades + reglas); la infraestructura implementa las interfaces que el dominio declara. La capa de dominio no conoce nada por debajo de ella.

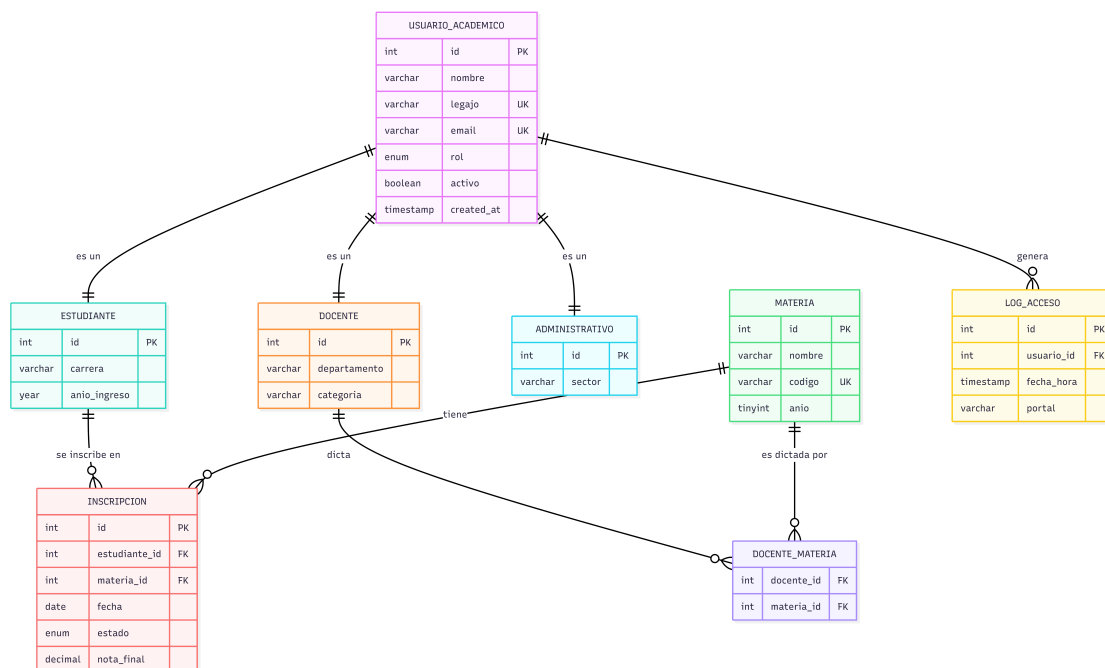


Figura 6: Diagrama UML de las entidades del dominio.

6. Estructura del Proyecto

```

1 src/main/java/universidad/
2 |-- Main.java           # Punto de entrada
3 |-- InputReader.java    # Lector de consola con validacion
4 |-- Validador.java      # Validadores estaticos reutilizables
5 |

```

```

6 |-- domain/
7 | |-- UsuarioAcademico.java      # Clase base (abstracta)
8 | |-- Estudiante.java           # Hereda UsuarioAcademico
9 | |-- Docente.java              # Hereda UsuarioAcademico
10 | |-- Administrativo.java        # Hereda UsuarioAcademico
11 | |-- Materia.java               # Entidad materia
12 | |-- Incripcion.java            # Entidad inscripcion
13 | |-- Nota.java                  # Value Object inmutable (0-10)
14 | |-- NegocioException.java      # Excepcion de dominio
15 | |-- Rol.java                   # Enum con id de DB
16 | |-- CategoriaDocente.java      # Enum con id de DB
17 | |-- EstadoIncripcion.java      # Enum con id de DB
18 | '-- repository/
19 | |-- IEstudianteRepository.java
20 | |-- IDocenteRepository.java
21 | |-- IAdministrativoRepository.java
22 | |-- IMateriaRepository.java
23 | '-- IIncripcionRepository.java
24 |
25 |-- application/
26 | |-- EstudianteService.java
27 | |-- DocenteService.java
28 | |-- AdministrativoService.java
29 | '-- IncripcionService.java
30 |
31 |-- infrastructure/
32 | |-- migration/
33 | | '-- Migrator.java           # Corre SQLs en orden al iniciar
34 | '-- persistence/
35 | |-- ConnectionFactory.java
36 | |-- EstudianteRepositoryJdbc.java
37 | |-- DocenteRepositoryJdbc.java
38 | |-- AdministrativoRepositoryJdbc.java
39 | |-- MateriaRepositoryJdbc.java
40 | '-- IncripcionRepositoryJdbc.java
41 |
42 '-- ui/menu/
43 | |-- MenuAlumnos.java
44 | |-- MenuDocentes.java
45 | '-- MenuAdministrativos.java
46
47 src/main/resources/db/migration/
48 |-- V1__schema.sql              # DDL: tablas normalizadas
49 '-- V2__seed.sql                 # Datos semilla
50
51 src/test/java/universidad/
52 '-- TestValidacionesYNota.java   # 25 tests de integracion

```

7. Detalle de Implementación

7.1. Capa de dominio: Value Object Nota

Nota es inmutable y valida en construcción. Dos notas con el mismo valor son iguales (por equals y hashCode), sin necesidad de identidad.

```
1 public final class Nota {
2
3     public static final double MINIMA = 0.0;
4     public static final double MAXIMA = 10.0;
5
6     private final double valor;
7
8     public Nota(double valor) {
9         if (valor < MINIMA || valor > MAXIMA) {
10             throw new IllegalArgumentException(
11                 "La nota debe estar entre " + MINIMA + " y " + MAXIMA +
12                 ".");
13         }
14         this.valor = valor;
15     }
16
17     public double getValor() { return valor; }
18
19     @Override
20     public boolean equals(Object o) {
21         if (this == o) return true;
22         if (!(o instanceof Nota otra)) return false;
23         return Double.compare(otra.valor, valor) == 0;
24     }
25
26     @Override
27     public int hashCode() { return Double.hashCode(valor); }
28
29     @Override
30     public String toString() { return String.valueOf(valor); }
```

Listing 1: Nota.java — value object inmutable.

7.2. Validación por campo con InputReader

Cada método de InputReader pide un valor y no retorna hasta que el usuario ingresa uno válido. Esto permite corregir campos uno a uno sin perder los ya ingresados.

```
1 public String seleccionarDeLista(String titulo, String prompt,
```

```
2         List<String> valores, Function<Integer, String> resolver) {
3         System.out.println(titulo);
4         for (int i = 0; i < valores.size(); i++) {
5             System.out.println("    " + (i + 1) + ". " + valores.get(i));
6         }
7         while (true) {
8             System.out.print(prompt);
9             String linea = scanner.nextLine().trim();
10            if (linea.isEmpty()) {
11                System.out.println("    Error: ingrese un numero o el valor.")
12            ;
13                continue;
14            }
15            int n;
16            try { n = Integer.parseInt(linea); }
17            catch (NumberFormatException e) { return linea; }
18            if (n < 1 || n > valores.size()) {
19                System.out.println("    Error: numero fuera de rango (1-"
20                    + valores.size() + ").");
21                continue;
22            }
23            return resolver.apply(n - 1);
24        }
```

Listing 2: InputReader.java — seleccion por lista numerada.

7.3. Persistencia con repositorios JDBC

Cada repositorio implementa una interfaz del dominio y encapsula las consultas SQL. Las transacciones que abarcan dos tablas (por ejemplo, insertar en `usuario_academico` + `estudiante`) se ejecutan dentro de un mismo `Connection` con `commit/rollback` manual.

```
1 @Override
2 public Long guardar(UsuarioAcademico usuario, String carrera, int anio)
3 {
4     try (Connection conn = ConnectionFactory.obtener()) {
5         conn.setAutoCommit(false);
6         try {
7             long id = insertarUsuario(conn, usuario, Rol.ESTUDIANTE);
8             insertarDetalle(conn, (int) id, carrera, anio);
9             conn.commit();
10            return id;
11        } catch (SQLException e) {
12            conn.rollback();
13            throw e;
14        }
15    }
```

```
14     } catch (SQLException e) {  
15         throw new NegocioException(  
16             "No se pudo guardar el estudiante: " + e.getMessage(), e);  
17     }  
18 }
```

Listing 3: EstudianteRepositoryJdbc — guardar con transaccion manual.

7.4. Migrador de base de datos

Migrator lista los archivos `.sql` ordenados por nombre y los ejecuta secuencialmente. Funciona tanto desde el classpath de desarrollo (`target/classes`) como desde el JAR empaquetado, detectando el protocolo de la URL (`file:` vs `jar:`).

7.5. Caso de uso: carga de nota

El método `DocenteService.cargarNota` concentra las reglas de negocio: construye un `Nota` (validación de rango), busca al estudiante por legajo, busca la materia por código o nombre, verifica que el alumno esté inscripto y finalmente persiste la nota.

```
1 public void cargarNota(String estudianteLegajo,  
2     String materiaCodigoONombre, double notaValor) {  
3     Nota nota = new Nota(notaValor);  
4  
5     var estudiante = repoEstudiantes.buscarPorLegajo(estudianteLegajo);  
6     if (estudiante == null) {  
7         throw new NegocioException(  
8             "El alumno con legajo '" + estudianteLegajo  
9             + "' no existe o no es estudiante.");  
10    }  
11  
12    Materia materia = repoMaterias  
13        .buscarPorCodigoONombre(materiaCodigoONombre)  
14        .orElseThrow(() -> new NegocioException(  
15            "La materia '" + materiaCodigoONombre + "' no existe."));  
16  
17    if (!repoInscripciones.existe(  
18        estudiante.getIdOrThrow(), materia.getId())) {  
19        throw new NegocioException(  
20            "El alumno '" + estudianteLegajo  
21            + "' no esta inscripto en la materia '"  
22            + materiaCodigoONombre + "'.");  
23    }  
24  
25    boolean ok = repoInscripciones.actualizarNotaFinal(  
26        estudiante.getIdOrThrow(), materia.getId(), nota.getValor());
```

```
27     if (!ok) {  
28         throw new NegocioException(  
29             "No se pudo guardar la nota. Intente nuevamente.");  
30     }  
31 }
```

Listing 4: DocenteService.java — metodo cargarNota.

8. Patrones de Diseño Aplicados

A continuación se enumeran los patrones y principios que el proyecto aplica de forma explícita, con una breve justificación de cada uno.

Arquitectura en capas (Clean Architecture)

El proyecto separa el código en cuatro capas con dependencias unidireccionales: la capa de dominio no importa ningún paquete de infraestructura ni de presentación. Esto permite cambiar la base de datos o la interfaz de usuario sin modificar las reglas de negocio.

SRP — Single Responsibility Principle

Cada clase tiene una única razón de cambio. Por ejemplo, `DocenteService` solo contiene casos de uso relacionados con docentes; la lógica de persistencia está en repositorios separados; la validación de entrada está en `InputReader` y `Validador`. Ninguna clase mezcla presentación con acceso a datos.

ISP — Interface Segregation Principle

En lugar de una interfaz genérica `IUsuarioRepository` que obligue a todos los repositorios a implementar métodos que no necesitan, el proyecto define cinco interfaces pequeñas y específicas: `IEstudianteRepository`, `IDocenteRepository`, `IAdministrativoRepository`, `IMateriaRepository` e `IInscripcionRepository`. Cada una expone solo las operaciones que su tipo de entidad requiere.

DIP — Dependency Inversion Principle

Los servicios de aplicación (`EstudianteService`, `DocenteService`, etc.) dependen de interfaces del dominio (`IEstudianteRepository`, `IMateriaRepository`), no de clases concretas como `EstudianteRepositoryJdbc`. La inyección se realiza manualmente en `Main.java`, que es el único punto que conoce las implementaciones concretas.

Value Object — Nota

Nota es inmutable, valida su invariante en el constructor (0–10) y su igualdad se define por valor, no por identidad. Dos instancias con el mismo número son iguales aunque sean objetos distintos. Esto evita que una nota inválida llegue a la base de datos y elimina la necesidad de validar el rango en cada punto de uso.

Patrón de identidad nula en entidades

Las entidades de dominio (**Estudiante**, **Docente**, etc.) tienen un campo `Integer id` que puede ser `null` cuando el objeto aún no fue persistido. El método `getIdOrThrow()` lanza una excepción si se intenta usar el `id` antes de guardar. Esto permite distinguir en el dominio entre “objeto nuevo” y “objeto recuperado de la base de datos” sin contaminar las entidades con lógica de persistencia.

Migrador de esquema

En lugar de depender de un ORM o de un script manual, la clase **Migrator** recorre los archivos `.sql` del classpath en orden alfabético y los ejecuta al iniciar la aplicación. Esto garantiza que la base de datos siempre tenga el esquema correcto, incluso la primera vez que se ejecuta.

Enum con mapeo a base de datos

Los enums `Rol`, `CategoriaDocente` y `EstadoInscripcion` incluyen un campo `id` que coincide con la clave primaria de la tabla de lookup correspondiente. Los métodos estáticos `desdeId(int)` y `desdeNombre(String)` permiten convertir desde la base de datos al enum y viceversa, encapsulando el mapeo en un solo lugar.

9. Instrucciones de Ejecución

9.1. Requisitos

- **Java 17** o superior (JDK).
- **Maven 3.6+** (gestor de dependencias y construcción).
- Sistema operativo: Windows, macOS o Linux con terminal.

9.2. Compilación

Desde la raíz del proyecto:

```
1 mvn clean package
```

Esto descarga la dependencia de SQLite, compila las 33 clases Java, ejecuta los tests de integración (*25 deben pasar*) y produce un JAR auto-contenido en `target/LSP_Universidad-1.0-SNAPSHOT`.

9.3. Ejecución

```
1 java -jar target/LSP_Universidad-1.0-SNAPSHOT.jar
```

La primera ejecución crea automáticamente el archivo `resources/sistema_universitario.db` (base SQLite) con el esquema normalizado y los datos semilla. El menú principal aparece en la terminal.

9.4. Ejecución de tests de integración

```
1 mvn dependency:build-classpath -Dmdep.outputFile=cp.txt
2 java -cp "target/classes:target/test-classes:$(cat cp.txt)" \
3     universidad.TestValidacionesYNota
```

10. Evidencias del Proyecto

Todo el material gráfico y audiovisual se encuentra en la carpeta `Recursos/` dentro del directorio raíz del proyecto.

10.1. Video de demostración de la aplicación

Muestra el ciclo completo: inicio, migración de base de datos, alta de alumno, inscripción a materia y carga de nota.

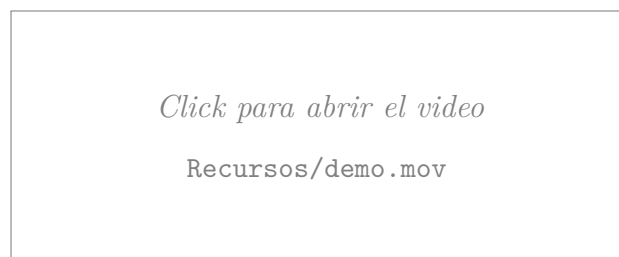


Figura 7: Video demostrativo de la aplicación en ejecución.

10.2. Video de recorrido por el código y estructura

Explica la organización en capas, los archivos principales y cómo se conectan entre sí.

Click para abrir el video

Recursos/demo-codigo-y-estructura.mov

Figura 8: Video de recorrido por el código fuente y la arquitectura.

10.3. Diagrama de arquitectura

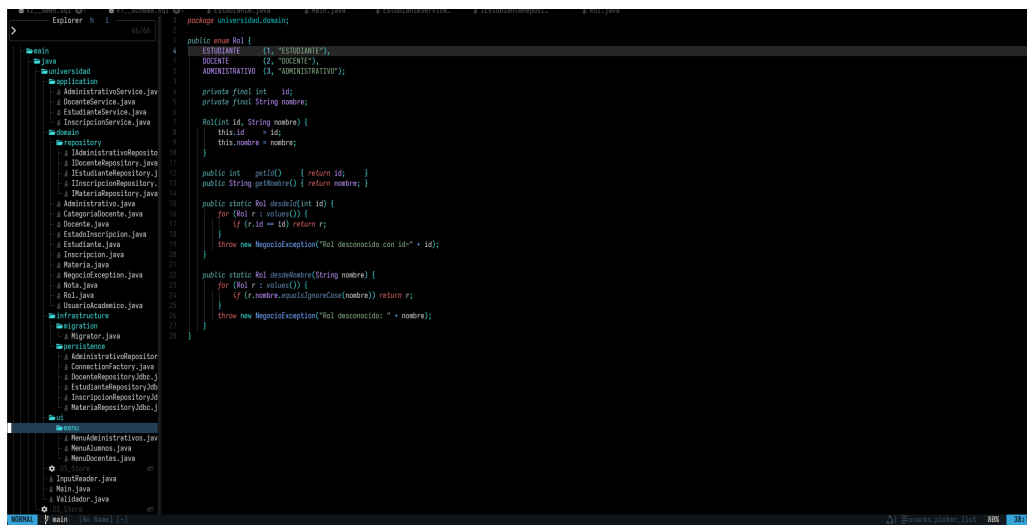


Figura 9: Diagrama de la estructura de capas y paquetes del proyecto.

10.4. Contenido de la carpeta Recursos/

Archivo	Contenido
demo.mov	Video: demostración de la aplicación funcionando.
demo-codigo-y-estructura.mov	Video: recorrido por el código fuente y la arquitectura.
estructura.png	Diagrama de la arquitectura en capas.
interaccion-app1.png	Captura: menú principal y listado de alumnos.
interaccion-app2.png	Captura: alta de un nuevo alumno.
interaccion-app3.png	Captura: inscripción de alumno a materia (selección por lista).
interaccion-app4.png	Captura: confirmación de inscripción registrada.
interaccion-app5.png	Captura: carga de nota (selección de alumno y materia).
interaccion-app6.png	Captura: confirmación de nota cargada y menú de docentes.

Cuadro 1: Listado del material en Recursos/.

11. Pruebas y Verificación

11.1. Compilación

El proyecto compila sin errores con `mvn clean package`. El *maven-shade-plugin* produce un fat JAR con la dependencia de SQLite embebida.

11.2. Test de integración

La clase `TestValidacionesYNota` ejecuta 25 casos de prueba que cubren:

- [1] **Validadores puros** (13 tests): email nulo, email sin arroba, email válido; campo vacío; año de ingreso fuera de rango; nota negativa, nota mayor a 10, nota válida.
- [2] **Constructores del modelo** (5 tests): nombre vacío, legajo vacío, email inválido, año inválido, construcción exitosa.
- [3] **Validaciones contra SQLite** (4 tests): legajo inexistente, materia inexistente, alumno no inscripto, carga exitosa.

- [4] **Persistencia real** (1 test): la nota cargada se lee correctamente desde otra instancia del repositorio.
- [5] **Listados** (2 tests): la base de datos semilla tiene al menos 2 estudiantes y 3 materias.

Resultado: **25/25 tests OK**.

11.3. Smoke test manual

Se ejecutó la aplicación con entrada simulada para verificar el flujo completo: 1 (Alumnos) → 1 (Listar) → 0 (Volver) → 0 (Salir). La aplicación listó los alumnos semilla correctamente y salió sin errores.

12. Conclusiones

El proyecto cumplió los objetivos planteados:

- Se obtuvo un **sistema de gestión universitaria funcional** en Java 17, con persistencia en SQLite, menú de consola interactivo y validación por campo.
- La **arquitectura en cuatro capas** demostró ser efectiva: el dominio está completamente aislado de JDBC y de la terminal; los servicios de aplicación contienen las reglas de negocio; la infraestructura es intercambiable sin tocar el dominio.
- La aplicación deliberada de los principios **SOLID** — en particular SRP, ISP y DIP — produjo un código con bajo acoplamiento y alta cohesión, donde cada clase tiene un propósito claro.
- El **esquema de base de datos normalizado** (3NF) con tablas de lookup, restricciones CHECK e índices sobre claves foráneas garantiza integridad referencial y evita datos inconsistentes.
- La **experiencia de usuario** se mejoró significativamente con listas numeradas desplegables para seleccionar alumnos y materias, eliminando la necesidad de memorizar legajos o códigos.

13. Trabajo Futuro

Quedan como líneas de mejora para versiones posteriores:

- **Interfaz gráfica:** reemplazar los menús de consola por una GUI con JavaFX o Swing, manteniendo la misma capa de dominio y aplicación.

- **Autenticación:** agregar un sistema de login con contraseñas hasheadas para que cada usuario acceda solo a su portal.
- **Consultas avanzadas:** historial académico del alumno (materias cursadas, aprobadas, promedio), reportes por docente y por carrera.
- **Cobertura de tests:** extender los tests de integración para cubrir los flujos de alta de docentes y administrativos, y agregar tests unitarios con mocks para los servicios de aplicación.
- **CI/CD:** configurar GitHub Actions para compilar, ejecutar los tests y publicar el JAR en cada push.